Case Study Work

Johan Nino Espino

2026-10-24

```
# 1. Load the original data
train_raw <- read.csv("data/data-train.csv")
test_raw  <- read.csv("data/data-test.csv")

train <- train_raw
test  <- test_raw

predictors <- c("St", "Re", "Fr")
raw_moments <- paste0("R_moment_", 1:4)
```

```
# 2. Check required columns
stopifnot(
  all(c(predictors, raw_moments) %in% names(train)),
  all(predictors %in% names(test))
)

# Check that the required columns are numeric and have no missing values.
# Fr = Inf is an intentional simulation setting, not a missing value.
stopifnot(
  all(vapply(train[c(predictors, raw_moments)], is.numeric, logical(1))),
  all(vapply(test[predictors], is.numeric, logical(1))),
  !anyNA(train[c(predictors, raw_moments)]),
  !anyNA(test[predictors])
)

# Re, St, and the raw moments must be finite.
stopifnot(
  all(is.finite(as.matrix(train[c("St", "Re", raw_moments)]))),
  all(is.finite(as.matrix(test[c("St", "Re")]))),
  all(train$Fr > 0),
```

```
  all(test$Fr > 0)
)
```

```
# 3. Convert raw moments into distribution summaries
m1 <- train$R_moment_1
m2 <- train$R_moment_2
m3 <- train$R_moment_3
m4 <- train$R_moment_4

train$mean_volume <- m1
train$variance_volume <- m2 - m1^2

# Stop and investigate if any variance is nonpositive.
if (any(train$variance_volume <= 0)) {
  stop("At least one calculated variance is nonpositive. Inspect those rows.")
}

train$sd_volume <- sqrt(train$variance_volume)

central_moment_3 <- m3 - 3 * m1 * m2 + 2 * m1^3
central_moment_4 <- m4 - 4 * m1 * m3 + 6 * m1^2 * m2 - 3 * m1^4

train$skewness_volume <- central_moment_3 / train$sd_volume^3
train$kurtosis_volume <- central_moment_4 / train$sd_volume^4

response_names <- c(
  "mean_volume", "variance_volume", "sd_volume",
  "skewness_volume", "kurtosis_volume"
)

stopifnot(
  all(is.finite(as.matrix(train[response_names])))
)
```

```
# 4. Print a compact preparation check
preparation_check <- data.frame(
  dataset = c("Training", "Test"),
  rows = c(nrow(train_raw), nrow(test_raw)),
  original_columns = c(ncol(train_raw), ncol(test_raw)),
  missing_entries = c(sum(is.na(train_raw)), sum(is.na(test_raw))),
  duplicate_input_settings = c(
    sum(duplicated(train_raw[predictors])),
```

```
    sum(duplicated(test_raw[predictors]))
  ),
  infinite_Fr_settings = c(
    sum(is.infinite(train_raw$Fr)),
    sum(is.infinite(test_raw$Fr))
  )
)

knitr::kable(preparation_check, caption = "Data preparation checks")
```

Table 1: Data preparation checks

dataset	rows	original_columns	missing_entries	duplicate_input_settings	infinite_Fr_settings
Training	89	7	0	0	30
Test	23	3	0	0	12

```
# Preview the predictors and calculated responses
knitr::kable(
  head(train[c(predictors, response_names)]),
  digits = 5,
  caption = "First six training observations"
)
```

Table 2: First six training observations

St	Re	Fr	mean_volume	variance_volume	sd_volume	skewness_volume	kurtosis_volume
0.10	224	0.052	0.00216	0.13035	0.36103	305.42800	93371.6555
3.00	224	0.052	0.00379	0.47041	0.68586	216.76225	47012.2515
0.70	224	Inf	0.00291	0.04349	0.20854	90.58974	8216.7778
0.05	90	Inf	0.06353	0.08662	0.29431	17.67980	420.2523
0.70	398	Inf	0.00037	0.00622	0.07889	257.58554	66372.7845
2.00	90	0.300	0.14780	1.98496	1.40888	12.64607	165.0999

```
# Summarize the calculated responses
summary(train[response_names])
```

```
  mean_volume       variance_volume       sd_volume         skewness_volume
 Min.   :0.000222   Min.   :1.000e-04   Min.   : 0.01006   Min.   : 11.97
 1st Qu.:0.002157   1st Qu.:2.447e-02   1st Qu.: 0.15643   1st Qu.: 72.55
```

```
 Median :0.002958   Median :8.078e-02   Median : 0.28423   Median :110.12
 Mean   :0.040394   Mean   :9.249e+01   Mean   : 3.65112   Mean   :162.81
 3rd Qu.:0.087868   3rd Qu.:5.268e-01   3rd Qu.: 0.72579   3rd Qu.:269.54
 Max.   :0.172340   Max.   :1.044e+03   Max.   :32.31526   Max.   :344.91
 kurtosis_volume
 Min.   :   150.5
 1st Qu.:  5622.3
 Median : 12158.7
 Mean   : 39749.6
 3rd Qu.: 72732.4
 Max.   :132136.7
```

```r
#| label: predictor-setup
#| message: false

library(ggplot2)

# Check that all test predictor levels occur in the training data
stopifnot(
  all(test$Re %in% train$Re),
  all(test$Fr %in% train$Fr),
  all(test$St %in% train$St)
)

# Use identical factor levels in training and test data
Re_levels <- sort(unique(train$Re))
Fr_levels <- sort(unique(train$Fr))
St_levels <- sort(unique(train$St))

train$Re_factor <- factor(train$Re, levels = Re_levels)
test$Re_factor  <- factor(test$Re, levels = Re_levels)

train$Fr_factor <- factor(train$Fr, levels = Fr_levels)
test$Fr_factor  <- factor(test$Fr, levels = Fr_levels)

# Inverse Froude number: Inf becomes 0
train$inv_Fr <- 1 / train$Fr
test$inv_Fr  <- 1 / test$Fr

# Compact summary of the original predictors
predictor_summary <- data.frame(
  Predictor = c("Re", "Fr", "St"),
```

```
  Training_levels = sapply(train[predictors], function(x) length(unique(x))),
  Test_levels = sapply(test[predictors], function(x) length(unique(x))),
  Training_min = sapply(train[predictors], min),
  Training_max = sapply(train[predictors], max),
  row.names = NULL
)

# Explicitly match the displayed predictor order
predictor_summary <- data.frame(
  Predictor = predictors,
  Training_levels = sapply(train[predictors], function(x) length(unique(x))),
  Test_levels = sapply(test[predictors], function(x) length(unique(x))),
  Training_min = sapply(train[predictors], min),
  Training_max = sapply(train[predictors], max),
  row.names = NULL
)

knitr::kable(
  predictor_summary,
  caption = "Predictor ranges and number of distinct settings"
)
```

Table 3: Predictor ranges and number of distinct settings

Predictor	Training_levels	Test_levels	Training_min	Training_max
St	14	13	0.050	3
Re	3	3	90.000	398
Fr	3	3	0.052	Inf

```
knitr::kable(
  data.frame(Fr = Fr_levels, Inverse_Fr = 1 / Fr_levels),
  digits = 4,
  caption = "Froude number and its reciprocal"
)
```

Table 4: Froude number and its reciprocal

Fr	Inverse_Fr
0.052	19.2308
0.300	3.3333

Fr	Inverse_Fr
Inf	0.0000

```
#| label: predictor-counts
#| fig-width: 7
#| fig-height: 3.5

# Combine predictors only; no response values are used here
design_data <- rbind(
  data.frame(train[predictors], Dataset = "Training"),
  data.frame(test[predictors], Dataset = "Test")
)

design_data$Dataset <- factor(
  design_data$Dataset,
  levels = c("Training", "Test")
)

# Helper function for count plots
plot_setting_counts <- function(variable, setting_levels, axis_label) {

  plot_data <- data.frame(
    Setting = factor(design_data[[variable]], levels = setting_levels),
    Dataset = design_data$Dataset
  )

  ggplot(plot_data, aes(x = Setting, fill = Dataset)) +
    geom_bar(position = "dodge") +
    scale_fill_manual(values = c("Training" = "#2166AC", "Test" = "#D6604D")) +
    labs(
      x = axis_label,
      y = "Number of simulations",
      fill = "Dataset"
    ) +
    theme_bw(base_size = 11) +
    theme(legend.position = "top")
}

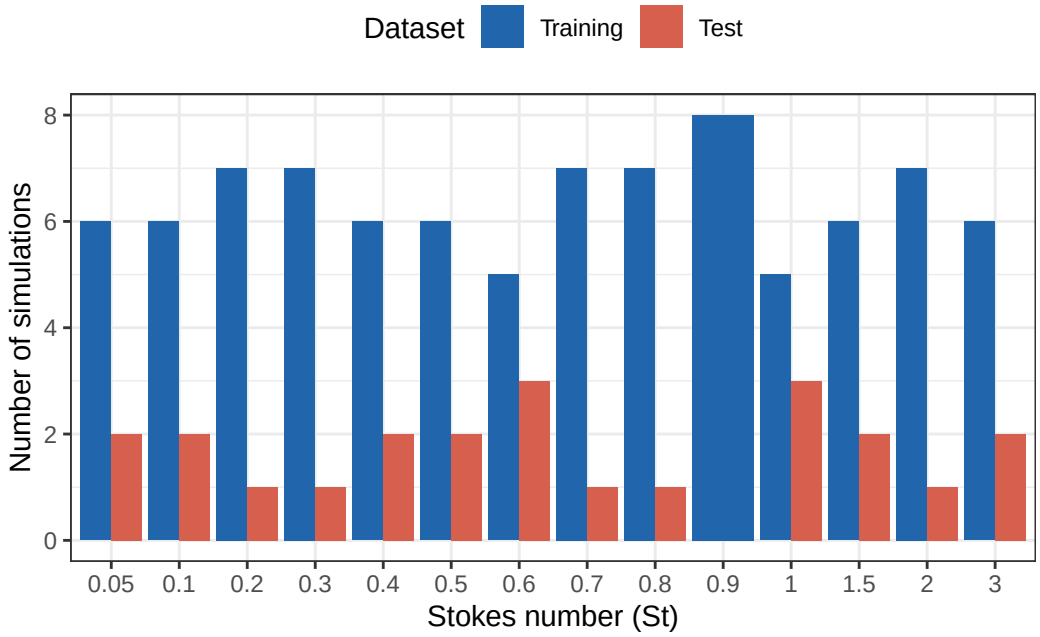
plot_setting_counts("Re", Re_levels, "Reynolds number (Re)")
```




```
plot_setting_counts("Fr", Fr_levels, "Froude number (Fr)")
```



```
plot_setting_counts("St", St_levels, "Stokes number (St)")
```




```
#| label: simulation-counts

# Include every combination, even combinations with no observations
design_counts <- expand.grid(
  Re = Re_levels,
  Fr = Fr_levels
)

design_counts <- design_counts[
  order(design_counts$Re, design_counts$Fr),
]

design_counts$Training <- mapply(
  function(re, fr) sum(train$Re == re & train$Fr == fr),
  design_counts$Re,
  design_counts$Fr
)

design_counts$Test <- mapply(
  function(re, fr) sum(test$Re == re & test$Fr == fr),
```

```
  design_counts$Re,
  design_counts$Fr
)

design_counts$Total <- design_counts$Training + design_counts$Test

knitr::kable(
  design_counts,
  row.names = FALSE,
  caption = "Simulation counts within each Re and Fr combination"
)
```

Table 5: Simulation counts within each Re and Fr combination

Re	Fr	Training	Test	Total
90	0.052	12	2	14
90	0.300	10	4	14
90	Inf	9	5	14
224	0.052	14	0	14
224	0.300	13	1	14
224	Inf	12	2	14
398	0.052	10	4	14
398	0.300	0	0	0
398	Inf	9	5	14

```
# Check for exact input settings shared by training and test data
shared_settings <- merge(
  unique(train[predictors]),
  unique(test[predictors]),
  by = predictors
)

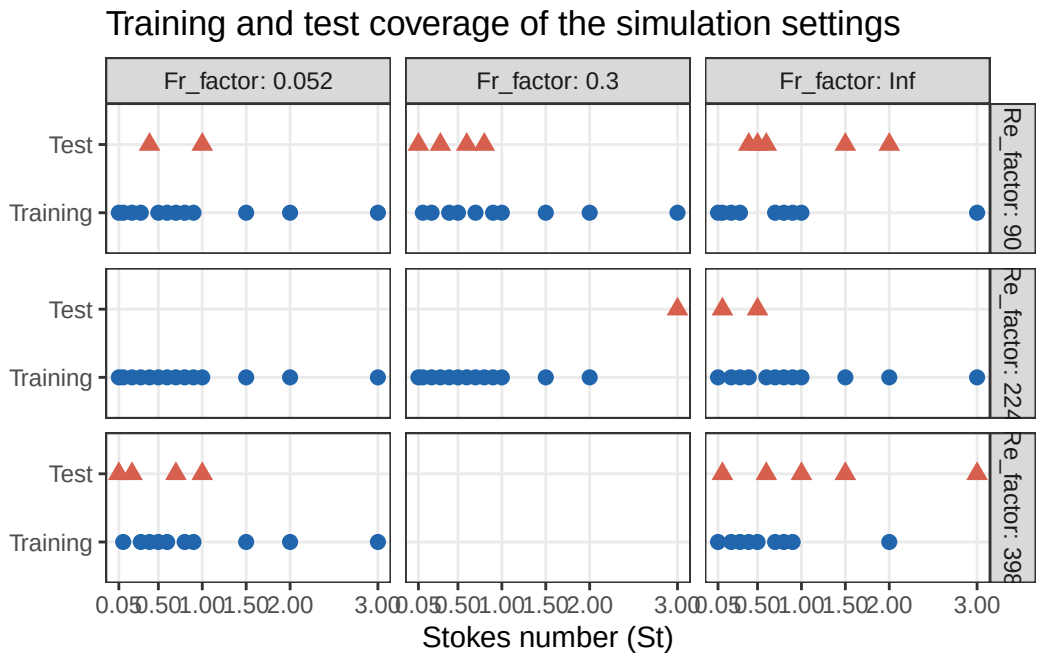
cat(
  "\nNumber of exact input settings shared by training and test:",
  nrow(shared_settings),
  "\n"
)
```

```
Number of exact input settings shared by training and test: 0
```

```r
#| label: simulation-coverage
#| fig-width: 8
#| fig-height: 6

design_data$Re_factor <- factor(design_data$Re, levels = Re_levels)
design_data$Fr_factor <- factor(design_data$Fr, levels = Fr_levels)

ggplot(
  design_data,
  aes(x = St, y = Dataset, color = Dataset, shape = Dataset)
) +
  geom_point(size = 2.5) +
  facet_grid(
    Re_factor ~ Fr_factor,
    labeller = label_both,
    drop = FALSE
  ) +
  scale_color_manual(
    values = c("Training" = "#2166AC", "Test" = "#D6604D")
  ) +
  scale_shape_manual(
    values = c("Training" = 16, "Test" = 17)
  ) +
  scale_x_continuous(
    breaks = c(0.05, 0.5, 1, 1.5, 2, 3)
  ) +
  labs(
    title = "Training and test coverage of the simulation settings",
    x = "Stokes number (St)",
    y = NULL
  ) +
  theme_bw(base_size = 11) +
  theme(
    legend.position = "none",
    panel.grid.minor = element_blank()
  )
```




```
#| label: response-plotting-data

response_columns <- c(
  "mean_volume",
  "sd_volume",
  "skewness_volume",
  "kurtosis_volume"
)

response_labels <- c(
  "Mean",
  "Standard deviation",
  "Skewness",
  "Kurtosis"
)

# Confirm that the responses exist and are finite
stopifnot(
  all(response_columns %in% names(train)),
  all(is.finite(as.matrix(train[response_columns]))))
)

# One row per simulation and response
```

```r
response_long <- do.call(
  rbind,
  lapply(seq_along(response_columns), function(j) {
    data.frame(
      St = train$St,
      Re_factor = train$Re_factor,
      Fr_factor = train$Fr_factor,
      Response = response_labels[j],
      Value = train[[response_columns[j]]]
    )
  })
)

response_long$Response <- factor(
  response_long$Response,
  levels = response_labels
)

# The log-scale plots below require positive values.
# Stop rather than silently removing nonpositive observations.
stopifnot(all(response_long$Value > 0))

# Consistent colors for Reynolds-number settings
Re_colors <- c(
  "90" = "#2166AC",
  "224" = "#1B7837",
  "398" = "#B2182B"
)
```

```r
#| label: response-distributions
#| fig-width: 8
#| fig-height: 5.5

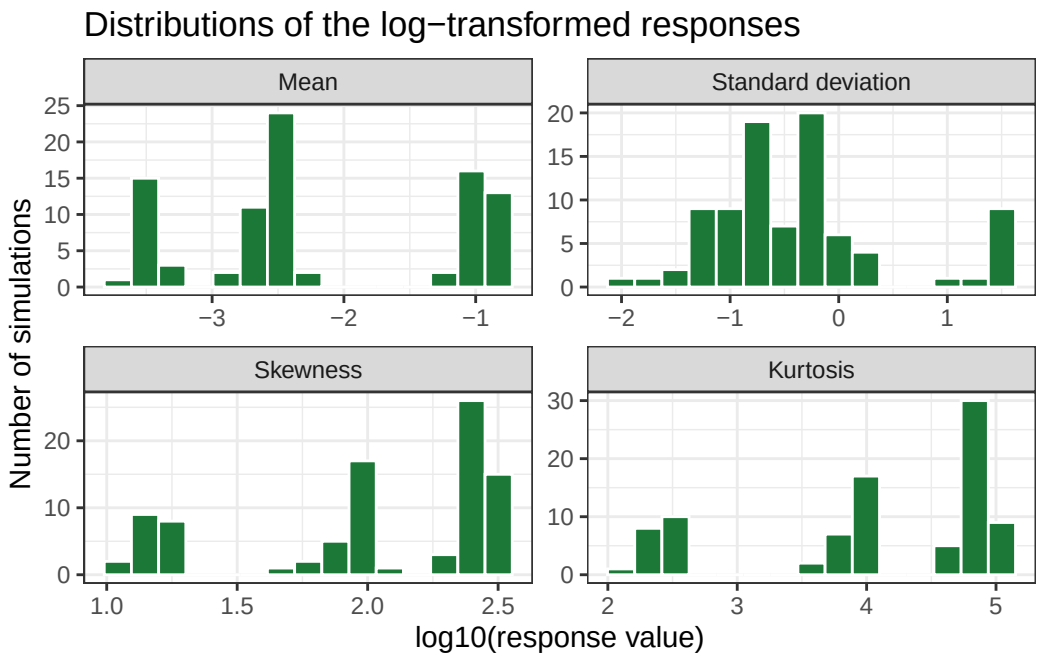
# Original-scale distributions
ggplot(response_long, aes(x = Value)) +
  geom_histogram(
    bins = 15,
    fill = "#2166AC",
    color = "white"
  ) +
  facet_wrap(
    ~ Response,
```

```
    scales = "free",
    ncol = 2
  ) +
  labs(
    title = "Distributions of the training responses",
    x = "Response value",
    y = "Number of simulations"
  ) +
  theme_bw(base_size = 11)
```




```
# Distributions after taking log10
ggplot(response_long, aes(x = log10(Value))) +
  geom_histogram(
    bins = 15,
    fill = "#1B7837",
    color = "white"
  ) +
  facet_wrap(
    ~ Response,
    scales = "free",
    ncol = 2
  ) +
```

```
  labs(
    title = "Distributions of the log-transformed responses",
    x = "log10(response value)",
    y = "Number of simulations"
  ) +
  theme_bw(base_size = 11)
```




```
#| label: responses-versus-St
#| fig-width: 8
#| fig-height: 3.8

for (response_name in response_labels) {

  plot_data <- response_long[
    response_long$Response == response_name,
  ]

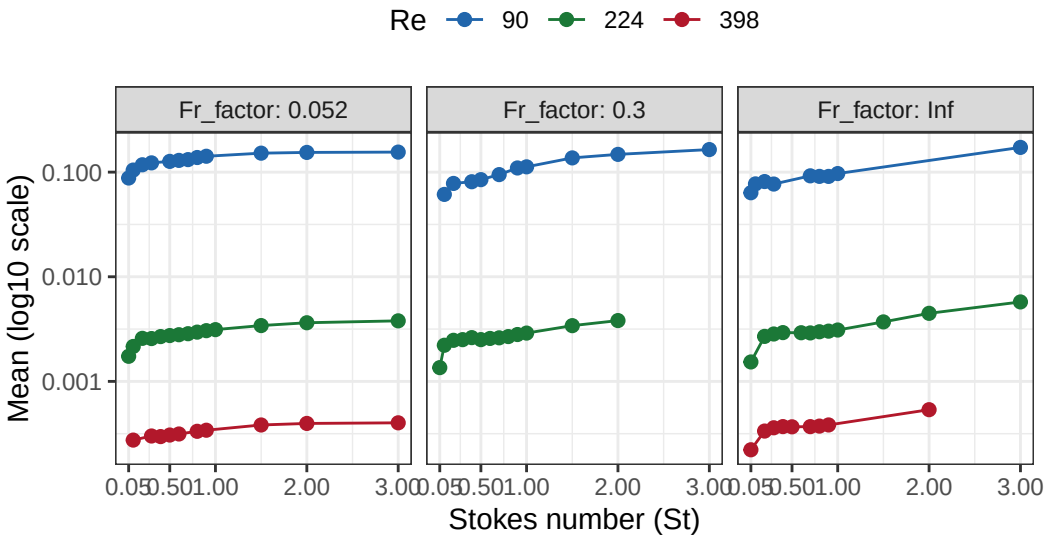
  p <- ggplot(
    plot_data,
    aes(
      x = St,
      y = Value,
```

```
      color = Re_factor,
      group = Re_factor
    )
  ) +
    geom_line(linewidth = 0.5) +
    geom_point(size = 2) +
    facet_wrap(
      ~ Fr_factor,
      nrow = 1,
      labeller = label_both,
      drop = FALSE
    ) +
    scale_color_manual(values = Re_colors) +
    scale_y_log10() +
    scale_x_continuous(
      breaks = c(0.05, 0.5, 1, 2, 3)
    ) +
    labs(
      title = paste(response_name, "versus Stokes number"),
      subtitle = "Lines connect available training observations within each Re-Fr regime",
      x = "Stokes number (St)",
      y = paste(response_name, "(log10 scale)"),
      color = "Re"
    ) +
    theme_bw(base_size = 11) +
    theme(legend.position = "top")

  print(p)
}
```

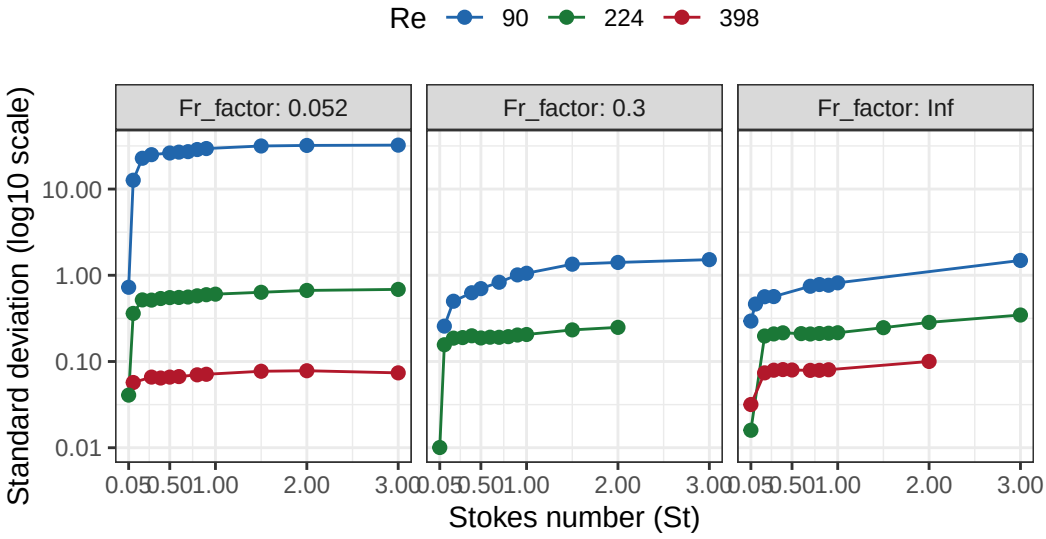
Mean versus Stokes number

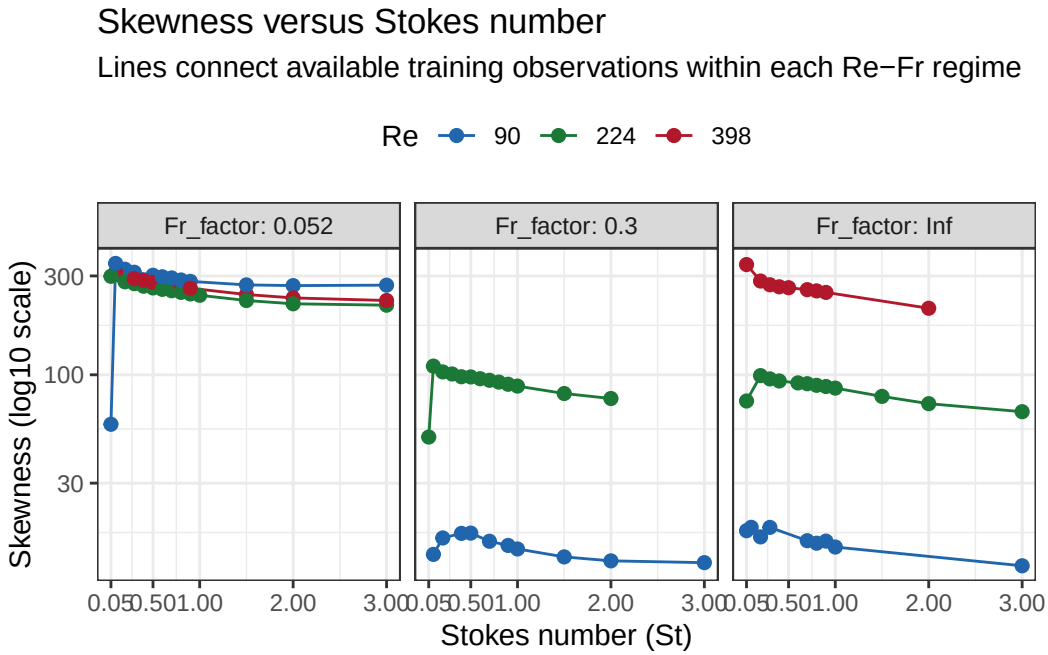
Lines connect available training observations within each Re−Fr regime

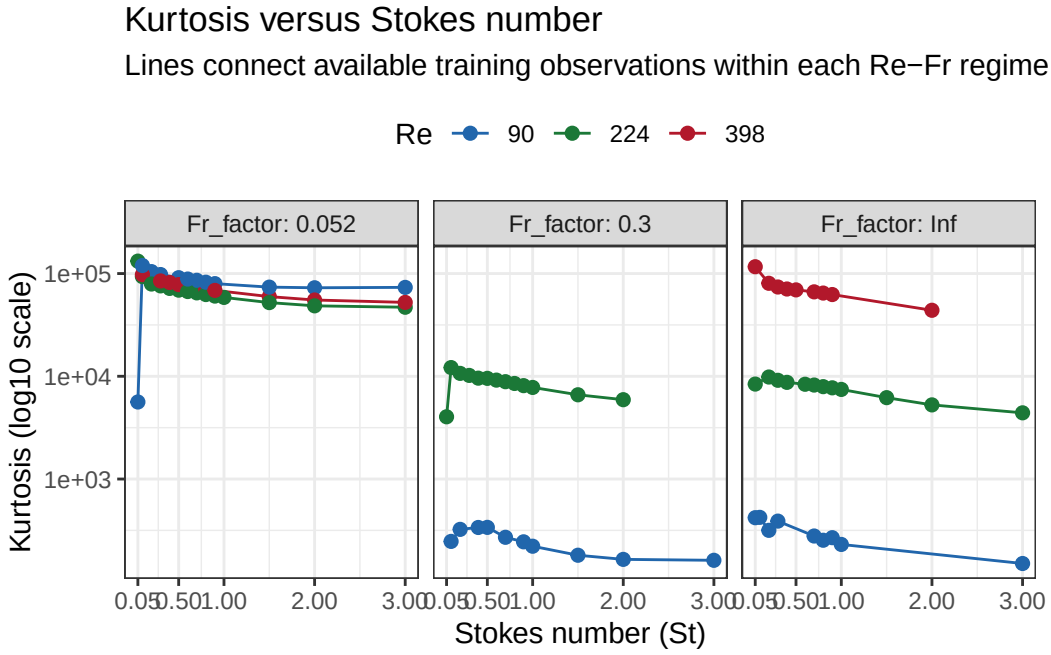


Standard deviation versus Stokes number

Lines connect available training observations within each Re−Fr regime

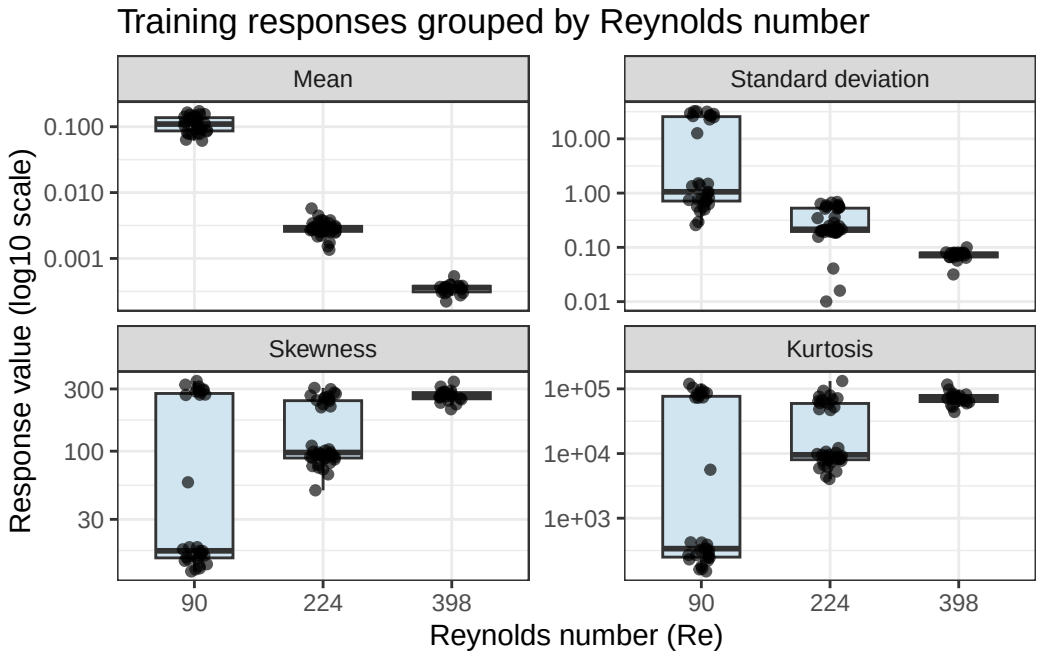







```
#| label: responses-by-Re-and-Fr
#| fig-width: 8
#| fig-height: 6
```

```r
# Responses grouped by Reynolds number
ggplot(
  response_long,
  aes(x = Re_factor, y = Value)
) +
  geom_boxplot(
    fill = "#D1E5F0",
    outlier.shape = NA,
    width = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.10,
      height = 0,
      seed = 325
    ),
    alpha = 0.65,
    size = 1.5
  ) +
  scale_y_log10() +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Training responses grouped by Reynolds number",
    x = "Reynolds number (Re)",
    y = "Response value (log10 scale)"
  ) +
  theme_bw(base_size = 11)
```




```
# Responses grouped by Froude number
ggplot(
  response_long,
  aes(x = Fr_factor, y = Value)
) +
  geom_boxplot(
    fill = "#D9F0D3",
    outlier.shape = NA,
    width = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.10,
      height = 0,
      seed = 325
    ),
    alpha = 0.65,
    size = 1.5
  ) +
  scale_y_log10() +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Training responses grouped by Froude number",
```

```
    x = "Froude number (Fr)",
    y = "Response value (log10 scale)"
  ) +
  theme_bw(base_size = 11)
```

